

Obisk Negotiation Agent

User guide - how to run it, what it does, and where people stay in the loop.

What it is

The agent negotiates creator sponsorship deals over email or DM. It has two halves. The calculator simulates what a creator's posts will deliver and turns that into three prices. The negotiator runs the conversation one message at a time, reading every dollar off those prices. An LLM (Claude) writes the outgoing messages and reads the incoming ones; it never invents a price and never picks the move. Every outbound message waits for a human approval until you switch autonomy on.

The three prices (the ladder)

Anchor. Where the agent opens. Low but credible - a calculated number, not a lowball.

Target. Where it aims. The fee that hits the campaign's ROI goal (default 3x) at expected delivery. The agent revises to here once if the creator pushes back, and no further.

Walk-away. The ceiling. Above this, an ordinary underperformance (a bottom-10% outcome) loses money. The agent never offers it and escalates asks far above it.

If the flat offer stalls, the agent restructures instead of overpaying: more videos at a fair per-video rate, so the total grows while the rate holds. Full math and the client-facing justification for every number: docs/CALCULATOR.md.

Setup

```
pip install -r requirements.txt
pip install anthropic          # optional, enables the LLM
set ANTHROPIC_API_KEY=...     # optional; omit to run offline
python -m pytest               # 67 checks, all offline
```

Without a key everything still runs: messages come from fixed templates and incoming text is read by keyword rules. With a key, Claude drafts each message from the live thread and reads real-world phrasing properly. Use the key for real work; offline mode is for tests and demos.

One file per campaign

A campaign is a YAML file (copy examples/unest_campaign.yaml). It is the only thing you edit per client:

brief: the facts the agent answers questions from - deliverables, timeline, usage rights, exclusivity, payment. Add keyword-keyed extra_facts (e.g. equity) to widen what it can answer without a human.

economics: conversion rate and LTV. Every price scales with these - calibrate them against real attribution before trusting absolute dollars.

negotiation: the stance - ROI target, opening aggressiveness, bundle size, the approval ceiling, call windows.

us_aliases: names, emails, or domains that identify your side of an email thread, so replay knows who is who.

Ways to run it

Talk to it yourself. You type as the creator, the agent replies live. This is the exact read-price-draft the platform runs on real Gmail threads - here you type the messages. The fastest way to see it is a real agent, not a canned demo.

```
py demo.py --chat
py demo.py --chat --campaign examples/unest_campaign.yaml --median 150000
```

See the calculation. Print the plain-English derivation of the three prices - reach, then dollars, then the ladder - so no number is a black box.

```
py demo.py --explain
```

Simulate. A rule-based creator plays the other side; you watch the full negotiation with approvals and the reasoning for each move. --random gives a fresh creator every run.

```
py demo.py --random
py demo.py --floor 2800 --opens ask --ask 3500
py demo.py --campaign examples/unest_campaign.yaml --count 20 --quiet
```

Replay a real thread. Paste a Gmail thread into a text file as-is (headers, signatures, quoted replies - the parser handles them). The agent shadows the deal: at each creator message you see how it read the message, what it would have done and why, next to what your team actually sent. Nothing is ever sent.

```
py demo.py --campaign examples/unest_campaign.yaml \
--replay examples/unest_thread.txt --median 150000
```

--median is the creator's typical per-video views, so the agent prices the same deal your team was pricing.

Spar against Claude. Claude role-plays a talent manager with a private playbook you write; the agent negotiates back. Use the transcript to judge the back-and-forth before trusting it live. Needs an API key.

```
py demo.py --spar "floor $2,600/video, opens at $4,500, concedes slowly"
```

Notebook. notebooks/ObIsk_Negotiation_Agent_Demo.ipynb walks the pricing model, a live negotiation, and the replay interactively.

Reading a decision

Every move the agent proposes shows the same four things: the action (open, revise, bundle, hold, accept, ...), the deal (total, videos, per-video rate), the expected ROI with its downside and upside, and a one-sentence rationale. If a line starting with NEEDS A HUMAN appears, the agent wants something from the team - read it, it says exactly what.

Where people stay in the loop

Approval gate. In the default mode a person approves every outbound message. In autonomous mode, anything above the campaign's dollar ceiling still requires sign-off.

ask_human. A question the brief cannot answer, or a reference to a call the agent was not on, pauses the

thread with a specific request. Answer it; the agent keeps the thread.

propose_call. When the creator wants a call, the agent proposes your call windows and flags a teammate to send the invite and run it. After the call, write what was agreed back into the thread - the agent cannot hear calls.

escalate_human. Extreme asks, exhausted rounds, and any contract-terms negotiation (exclusivity, equity, kill fees) hand the whole thread to a person. The agent never negotiates paper.

hold_firm. Not a handoff, but worth knowing: a follow-up with no counter gets the standing offer restated. The agent never bids against itself.

Plugging into the Oblsk platform

The platform already polls Gmail threads, classifies messages, and runs the review UI. The agent plugs in as an HTTP service that suggests the next reply:

```
py -m oblisk_negotiator.service --campaign examples/unest_campaign.yaml --port 8788
```

POST /suggest takes the thread and the creator's view numbers; the response has the drafted message, the move, the ladder, whether it needs approval, and any task for a human. The service is stateless: state is derived from the thread itself, so offers a person made count as the agent's positions and it picks threads up mid-flight. Wiring details and the exact request/response shapes: docs/INTEGRATION.md.

Walk-away vs. a creator's floor

Walk-away is OUR ceiling, computed from the numbers: above it, an ordinary bottom-10% outcome loses money. A creator's floor is THEIR private minimum - the agent never sees it. In demos, the 'sim creator's hidden floor' line is a parameter of the fake counterpart, shown so you can judge the run; real threads have no such input. When target and walk-away print as the same number, the creator's views are so volatile that the fee hitting the ROI goal would lose money on a bad outcome - the ceiling clamps the target, and the printout says so.

Environment variables

ANTHROPIC_API_KEY - enables the LLM (drafting, reading, sparring).

OBLISK_LLM_MODEL - model override; defaults to claude-opus-4-8.

OBLISK_NO_LLM=1 - force offline mode even if a key is set.

If something looks wrong

Prices too low or too high across the board: fix economics in the campaign file - every number is derived from conversion x LTV. A message read incorrectly in replay: you are probably offline; the keyword reader is approximate, the LLM reader is not. A thread stopped: look for the NEEDS A HUMAN line; the agent is waiting on you, not stuck.